

Interpolation-based Transition Invariant Generation for Proving and Disproving Termination

Konstantin Britikov

Martin Blicha

Grigory Fedyukovich

Natasha Sharygina

Abstract—Termination and nontermination of infinite-state systems are complementary problems that, despite their close connection, are typically addressed by separate techniques. The core idea of this paper is to connect termination and nontermination analysis, enabling the two to share their intermediate results and guide one another. We present a new termination analysis approach based on the generation of well-founded transition invariants. It leverages Craig interpolation for transition invariant generation, capturing the structural reasons for termination. The proposed technique extends safety-based nontermination analysis, allowing it to prove both termination and nontermination within a unified framework. We implemented our approach in the Golem verification framework and evaluated it on benchmarks from the Termination Competition (TermComp). Empirical results demonstrate that combining termination and nontermination is beneficial and yields performance comparable to state-of-the-art tools.

I. INTRODUCTION

Termination analysis of infinite-state systems is a fundamental problem that necessitates the discovery of complex ranking functions or inductive invariants to over-approximate system behavior. Proving termination and nontermination is undecidable [8], yet there exist many techniques that succeed on a large class of real-world problems. Modern techniques treat termination and nontermination as separate problems, since each has a different proof procedure. The existence of a ranking function by e.g. [1], [2], [16], [18], [20], [23], [29], [31], [32], [35] witnesses termination by providing a measure that strictly decreases along every execution. Alternatively, fixpoint computation, e.g. [34], and disjunctively well-founded transition invariants [3], [14], [22], [33] have been proposed for termination analysis. Nontermination techniques are mostly based on the detection of recurrent sets — sets of states from which execution cannot escape [1], [9], [10], [12], [16], [17], [24]. There also exist alternative approaches based on Hoare-style reasoning [25], safety proving [4], [12], representing infinite runs as sums of geometric series [26], or synthesis of *negative* exact loop bounds [31].

In this work, we present a unified framework for proving both termination and nontermination. This framework extends the safety-based nontermination analysis [12] (SNA), enabling it to prove termination. The extension is a novel interpolation-based method for generating disjunctively well-founded transition invariants for termination analysis. The proposed approach uses terminating transition traces constructed by SNA, overapproximating them with a Craig interpolation [15]. The interpolant is processed into a transition invariant candidate, which is used to prove termination. Additionally, based on the transition invariant candidate, algorithm creates smaller,

focused (non)termination problems, which guide the analysis of the whole transition system. Solving these problems helps to extend the transition invariant candidate and enables an efficient nontermination analysis of complex problems.

We implemented and evaluated our new technique in the Golem Constrained Horn Clause solver [5]. We evaluated it on the Integer Transition Systems benchmarks from the Termination Competition [19]. Our tool solved a large number of terminating instances and demonstrated significant improvement for nontermination analysis, compared to the classical SNA. The comparison with the winners of Termination Competition 2025 (KoAT, LoAT [27], T2 [9]) shows that our approach is competitive with the state-of-the-art tools, and is able to solve unique instances, including those that were never solved by any tool before.

Overall, the contributions of this paper are as follows: (1) a novel interpolation-based method for the generation of disjunctively well-founded transition invariants, (2) a unified termination/nontermination analysis procedure that utilises the intermediate results of (non)termination analysis for self-guidance and (3) its implementation and successful evaluation.

II. PRELIMINARIES

This section introduces the definitions and formalisms used in the paper. Logical formulas in our approach are defined over the theory of Linear Integer Arithmetic (LIA). We use X, X' to denote sets of variables (unprimed and primed), where X is a set of state variables, and $X' = \{x' | x \in X\}$ is a set of next state variables. If a formula contains variables over multiple states, we use superscript notation, e.g., $X^{(0)}, \dots, X^{(n)}$.

The formula defined over X is a state formula, and a formula over $X \cup X'$ is a transition formula. State formulas encode a set of states, and transition formulas define binary relations over the states. We use $Id(X, X')$ to denote the identity transition formula $\bigwedge_{x \in X} x = x'$.

Transition systems and Safety Problem: A transition system is defined as $TS = \langle Init, Tr, X \rangle$, where $Init$ is a formula encoding the initial states of the system, Tr is a transition formula, encoding the transition relation, and X is the set of system variables. With respect to transition system, a *transition trace* is a formula encoding multiple consecutive transitions: $Tr^n(X^{(0)}, \dots, X^{(n)}) = Tr(X^{(0)}, X^{(1)}) \wedge \dots \wedge Tr(X^{(n-1)}, X^{(n)})$. A *safety problem* is $P = \langle Init, Tr, Bad, X \rangle$, where Bad is a formula representing states that violate the safety property.

Safety verification amounts to determining whether there exists a transition trace that reaches a *Bad* state from

some initial state. If TS is safe, then there exists an inductive invariant $Inv(X)$ such that (1) $\forall X. Init(X) \rightarrow Inv(X)$, (2) $\forall X, X'. Inv(X) \wedge Tr(X, X') \rightarrow Inv(X')$, and (3) $\forall X. Inv(X) \rightarrow \neg Bad(X)$. We use **SAFETY-CHECK** (P) to denote a safety verification procedure that returns a tuple $\langle \text{SAFE}, Inv(X) \rangle$ if TS is safe ($Inv(X)$ is an inductive invariant). If TS is unsafe it returns $\langle \text{UNSAFE}, Tr^n(X^{(0)}, \dots, X^{(n)}) \rangle$, s.t. $\exists X^{(0)}, \dots, X^{(n)}$ ¹.

$$Init(X^{(0)}) \wedge Tr^n(X^{(0)}, \dots, X^{(n)}) \wedge Bad(X^{(n)})$$

Nontermination: A transition system is called **nonterminating** if there exists an initial state from which it is possible to take any number of transitions, meaning $\exists X. Init(X)$ such that:

$$\forall n \in \mathbb{N}. \exists X^{(1)}, \dots, X^{(n)}. Tr^n(X, X^{(1)}, \dots, X^{(n)})$$

Definition 1 (Recurrent Set). For a Transition System $\langle Init, Tr, X \rangle$, a formula $N(X)$ is a recurrent set if: (1) $\forall X. N(X) \rightarrow \exists X'. Tr(X, X') \wedge N(X')$, (2) $\exists X. N(X) \wedge Init(X)$.

Transition System is nonterminating if there exists a *recurrent set*. Intuitively, every state in the recurrent set has a successor that also lies in the recurrent set, and the recurrent set contains some of the initial states.

Termination: A TS is called **terminating** if for every initial state it is possible to take only a finite number of transitions. Transition System $\langle Init, Tr, X \rangle$ terminates, if $\forall X. Init(X)$ the following holds:

$$\exists n \in \mathbb{N}. \neg \exists X^{(1)}, \dots, X^{(n)}. Tr^n(X, X^{(1)}, \dots, X^{(n)})$$

Definition 2 (Transition invariant). For a Transition System $\langle Init, Tr, X \rangle$, $TrInv(X, X')$ is a transition invariant if: $\forall X, X'. R(X) \wedge Tr^+(X, X') \rightarrow TrInv(X, X')$, where $Tr^+(X, X')$ is the transitive closure of $Tr(X, X')$ and $R(X)$ is a formula encoding all states reachable in TS .

Definition 3 (Well-founded Relation). A binary relation $R \subseteq D \times D$ over a domain D is *well-founded* if there exists no infinite descending chain:

$$\nexists d_0, d_1, d_2, \dots \in D. \forall i \in \mathbb{N}. (d_i, d_{i+1}) \in R$$

A relation $R(X, X')$ can be proven [29] to be well-founded by synthesizing a ranking function $f(X)$, such that $\forall X, X'. R(X, X') \rightarrow f(X) > f(X')$.

A transition invariant can be used to prove termination [30]. If a transition invariant is a finite union of well-founded relations (for example, if each relation admits a ranking function), then the transition system terminates.

Definition 4 (Disjunctively well-founded relation). A relation $W(X, X') = \bigcup_{i=0}^n R_i(X, X')$ is called disjunctively well-founded if every relation $R_i(X, X')$ is well-founded.

¹It is a shorthand notation for $\exists X^{(0)}, \dots, \exists X^{(n)}$

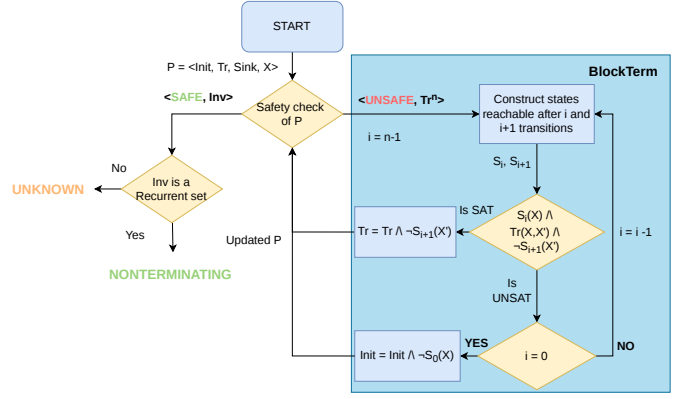


Figure 1. Execution flow of Safety-based Nontermination analysis, cf. [12]

Craig Interpolation: Given two formulas A and B where $A \wedge B$ is unsatisfiable, a Craig interpolant [15] for A and B is a formula I , where: (1) $A \rightarrow I$; (2) $I \wedge B$ is unsatisfiable; and (3) I only contains the symbols, common to both A and B . Craig interpolant is used in our approach as an abstraction technique, that can overapproximate formulae. We use $ITP(A, B)$ to denote the computation of the Craig interpolant.

A. Proving Nontermination via Safety

Many techniques reduce termination and nontermination analysis to safety verification. One notable approach is the reduction of nontermination to safety, which constructs recurrent sets via inductive invariants [12]. It was introduced in the context of proving nontermination of programs with loops, and can be extended to Transition Systems.

Definition 5 (Sink states). For a Transition System $\langle Init, Tr, X \rangle$, a state s is a sink state, if $\neg \exists s'. Tr(s, s')$. We use $Sink(X)$ to denote the formula encoding all sink states in the transition system: $\forall X. Sink(X) \leftrightarrow \neg \exists X'. Tr(X, X')$

Algorithm 1 for Safety-based Nontermination analysis (SNA) takes a safety problem $P = \langle Init, Tr, Sink, X \rangle$, where sink states are constructed from TS using quantifier elimination (QE): $Sink(X) = \neg QE(\exists X'. Tr(X, X'))$. As an output, it returns **NONTERM** if the system is nonterminating, and **UNKNOWN**, if the algorithm cannot conclude the analysis. Figure 1 contains the execution flow of Safety-based Nontermination.

SNA iteratively solves increasingly refined safety problems until the given sink states are proven unreachable. In every iteration, if there exists a trace reaching sink states (line 3), **BLOCKTERM** (Algorithm 2) refines the safety problem P by blocking the states that deterministically lead to the sink states, updating the safety problem. **BLOCKTERM** is described in detail below. When the original sink states are no longer reachable, SNA checks if the computed inductive invariant $Inv(X)$ is a recurrent set (line 6). This amounts to deciding satisfiability of the following formula:

$$Inv(X) \wedge \neg \exists X'. Tr(X, X').$$

Algorithm 1: Safety-based Nontermination Analysis (SNA), cf. [12]

Input : Safety problem $P = \langle \text{Init}, \text{Tr}, \text{Sink}, X \rangle$
Output: NONTERM | UNKNOWN
1 **while** $\top$ **do**
2 $\langle \text{res}, \phi = \text{Tr}^n \text{ or } \text{Inv} \rangle \leftarrow \text{SAFETYCHECK}(P)$
3 **if** $\text{res} = \text{UNSAFE}$ **then**
4 $P \leftarrow \text{BLOCKTERM}(P, \phi)$
5 **else**
6 **if** $\forall X. \phi(X) \rightarrow \exists X'. \text{Tr}(X, X')$ **then**
7 **return** NONTERM
8 **return** UNKNOWN

Algorithm 2: BLOCKTERM (P, Tr^n), cf. [12]

Input : Safety problem $P = \langle \text{Init}, \text{Tr}, \text{Sink}, X \rangle$,
terminating trace $\text{Tr}^n(X^{(0)}, \dots, X^{(n)}) =$
 $\text{Tr}(X^{(0)}, X^{(1)}) \wedge \dots \wedge \text{Tr}(X^{(n-1)}, X^{(n)})$
Output: Refined safety problem P'
Data : $S_0, \dots, S_n$ - formulas which encode states within
trace reachable in 0, ..., n transitions
1 $i \leftarrow n - 1$
2 **while** $i > 0$ **do**
3 $S_{i+1}(X) \leftarrow \text{QE}(\exists X^{(0)}, \dots, X^{(i)}, X^{(i+2)}, \dots, X^{(n)}.$
 $\text{Init}(X^{(0)}) \wedge \text{Tr}^n \wedge \text{Sink}(X^{(n)})) [X^{(i+1)} \mapsto X]$
4 $S_i(X) \leftarrow \text{QE}(\exists X^{(0)}, \dots, X^{(i-1)}, X^{(i+1)}, \dots, X^{(n)}.$
 $\text{Init}(X^{(0)}) \wedge \text{Tr}^n \wedge \text{Sink}(X^{(n)})) [X^{(i)} \mapsto X]$
5 **if** $\text{SAT} ? S_i(X) \wedge \text{Tr}(X, X') \wedge \neg S_{i+1}(X')$ **then**
6 $\text{Tr}'(X, X') \leftarrow \text{Tr}(X, X') \wedge \neg S_{i+1}(X')$
7 **return** $\langle \text{Init}, \text{Tr}', \text{Sink}, X \rangle$
8 $i \leftarrow i - 1$
9 $\text{Init}'(X^{(0)}) \leftarrow \text{Init}(X^{(0)}) \wedge$
 $\neg \text{QE}(\exists X^{(1)}, \dots, X^{(n)}. \text{Init}(X^{(0)}) \wedge \text{Tr}^n \wedge \text{Sink}(X^{(n)}))$
10 **return** $\langle \text{Init}', \text{Tr}, \text{Sink}, X \rangle$

If $\text{Inv}(X)$ is a recurrent set, then the transition system is nonterminating. Otherwise, SNA returns UNKNOWN.

The BLOCKTERM procedure takes as input a safety problem P and a trace formula $\text{Tr}^n(X^{(0)}, \dots, X^{(n)})$ such that $\text{Init}(X^{(0)}) \wedge \text{Tr}^n(X^{(0)}, \dots, X^{(n)}) \wedge \text{Sink}(X^{(n)})$ is satisfiable. The algorithm iteratively analyses the trace from the last transition backwards. Using QE, Algorithm 2 computes S_i and S_{i+1} , encoding the states that are reachable after i and $i + 1$ transitions from Init (lines 3, 4). Then, at line 5 it checks if $S_i(X) \wedge \text{Tr}(X, X') \wedge \neg S_{i+1}(X')$ is satisfiable. Intuitively, this query checks if it is possible to take an alternative transition at step i , one that does not lead to the sink states. If it is possible, the algorithm blocks the states deterministically leading to the sink states by excluding them from the transition relation (line 6). If the whole trace is deterministic (it is not possible to take an alternative transition along the trace), BLOCKTERM blocks the subset of initial states deterministically leading to sink (line 9).

Lemma 1. *If Algorithm 1 returns NONTERM, then the transition system is nonterminating, cf. [12].*

Example 1. Consider a transition system $TS = \langle \text{Init}, \text{Tr}, X \rangle$ with $X = \{x\}$, $\text{Init}(X) \triangleq \top$, and $\text{Tr}(X, X') \triangleq (x' = x -$

$1 \vee x' = x - 2) \wedge x \neq 0$. The sink states are $\text{Sink}(X) \triangleq x = 0$, and the initial safety problem is $P = \langle \text{Init}, \text{Tr}, \text{Sink}, x \rangle$. This TS is nonterminating, since x infinitely decreases.

The execution of Algorithm 1 updates initial states and transition in two iterations of the algorithm, via BLOCKTERM $\text{Init}(X) \leftarrow x \neq 0$, and $\text{Tr}(X, X') \leftarrow \text{Tr}(X, X') \wedge x' \neq 0$. The safety check of the updated problem concludes SAFE, with $\text{Inv}(X) \triangleq x \neq 0$. $\text{Inv}(X)$ is a recurrent set, and algorithm returns NONTERM.

Motivation for transition invariant-based termination analysis. Consider a small modification (replacing $x \neq 0$ with $x > 0$): $\text{Tr}(X, X') \triangleq (x' = x - 1 \vee x' = x - 2) \wedge x > 0$. This renders TS terminating, since x is strictly positive and decreases every step. However, Algorithm 1 cannot detect termination by design, and it would return UNKNOWN.

Let us examine the execution of Algorithm 1 on this example. First, using BLOCKTERM, it blocks initial states that instantly terminate: $\text{Init}(X) \leftarrow x > 0$. In the second iteration, SAFETYCHECK(P) returns UNSAFE with a trace $\text{Tr}^1(X^{(0)}, X^{(1)})$, detecting that $\exists X^{(0)}, X^{(1)}. \text{Init}(X^{(0)}) \wedge \text{Tr}^1(X^{(0)}, X^{(1)}) \wedge \text{Sink}(X^{(1)})$. Our key insight is to generalise such terminating traces into a formula describing an entire class of terminating behaviours. If this generalisation yields a disjunctively well-founded transition invariant over all reachable states, it witnesses termination. Particularly, $\text{TrInv}(X, X') \triangleq x' < x \wedge x > 0$ is a well-founded transition invariant that proves the termination of TS . Section III presents the algorithm that automates this generalisation via Craig interpolation.

III. TERMINATION ANALYSIS VIA INTERPOLATION-BASED TRANSITION INVARIANT GENERATION

We present a new approach to proving termination that leverages safety verification to generate a well-founded transition invariant via interpolation. At each iteration, our procedure constructs a trace from the initial states to a sink state. A candidate well-founded transition invariant is then constructed based on the overapproximation of the terminating trace. If the candidate is valid for all reachable states (meaning it is a transition invariant), termination is concluded. The interpolation-based approach enables efficient termination condition-guided transition invariant generation.

A. Interpolation-based Generation of Well-Founded Transition Invariants

The core of our approach is TRINVGEN, a procedure for the automated interpolation-based generation of well-founded transition invariants. Given a transition system and a terminating trace, it constructs a disjunctively well-founded approximation of the trace. If the resulting formula constitutes a transition invariant over all reachable states, it witnesses termination.

The procedure consists of five steps: (1) Computing NT^n , the formula encoding states from which non-sink states can be reached within n transitions. NT^n is used to construct

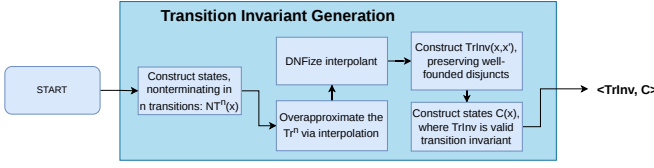


Figure 2. Execution Flow of Interpolation-based Transition Invariant Generation

Algorithm 3: $\text{TRINVGEN}(P, Tr^n, TrInv)$.

Input : Safety problem $P = \langle Init, Tr, Sink, X \rangle$, transition trace $Tr^n(X^{(0)}, X^{(1)}, \dots, X^{(n)})$, candidate transition invariant $TrInv(X, X')$

Output: Updated transition invariant $TrInv(X, X')$ and formula $C(X)$ encoding states covered by transition invariant

- 1 $NT^n(X^{(0)}) \leftarrow \text{QE}(\exists X^{(1)}, \dots, X^{(n)}).$
 $Tr^n(X^{(0)}, X^{(1)}, \dots, X^{(n)}) \wedge \neg Sink(X^{(n)})$
- 2 $itp(X, X') \leftarrow$
 $\text{ITP}(Tr^n(X, \dots, X'), \neg NT^n(X) \wedge \neg Sink(X'))$
- 3 $Cs(X, X') \leftarrow \text{ToDNF}(itp)$
- 4 $TrInv \leftarrow TrInv \vee \bigvee_i \{d_i \in Cs \mid \exists \text{ ranking function } f(X)\}$
- 5 $C(X) \leftarrow \neg \text{QE}(\exists X', X''). (Id(X, X') \vee TrInv(X, X')) \wedge$
 $Tr(X', X'') \wedge \neg TrInv(X, X'')$
- 6 **return** $\langle TrInv, C \rangle$

the interpolant. The negation of NT^n encodes the states from which $Sink$ is guaranteed to be reached in at most n transitions. (2) Constructing an over-approximation of the terminating trace via interpolation. The usage of sink states allows the interpolant to capture the reason for termination. (3) Converting the interpolant into an underapproximating DNF formula. This enables efficient ranking function synthesis for each disjunct. (4) Filtering disjuncts, preserving only disjuncts that encode well-founded relations. These disjuncts are used to construct a candidate transition invariant. Only well-founded relations are preserved, as transition invariant needs to be disjunctively well-founded to witness termination. (5) Computing the formula $C(X)$, encoding states for which candidate transition invariant is valid. Transition invariant needs to be valid only over the reachable states to conclude termination. Therefore, if all reachable states satisfy $C(x)$, then TS is terminating. The overall flow of the procedure is illustrated in Figure 2 and described in Algorithm 3.

Algorithm 3 takes as input a safety problem $P = \langle Init, Tr, Sink, X \rangle$, a transition trace $Tr^n(X^{(0)}, X^{(1)}, \dots, X^{(n)})$, and a transition invariant formula $TrInv(X, X')$. It returns a tuple of formulas $\langle TrInv, C \rangle$, where $TrInv(X, X')$ is a disjunctively well-founded transition invariant candidate, and $C(X)$ is a formula encoding the states for which $TrInv$ is a valid transition invariant. It proceeds in the following steps:

Step 1: Computing NT^n (line 1). $NT^n(X)$ encodes all states from which a non-sink state is reachable within n transitions. The following property defines it: $\forall X^{(0)}. NT^n(X^{(0)}) \leftrightarrow$

$$\exists X^{(1)}, \dots, X^{(n)}. Tr^n(X^{(0)}, \dots, X^{(n)}) \wedge \neg Sink(X^{(n)})$$

NT^n is constructed with quantifier elimination, eliminating all variables from the formula except for $X^{(0)}$. Construction of NT^n enables the overapproximation of the trace.

Example 2. Consider $TS = \langle Init, Tr, X \rangle$ with $Init \triangleq x > 0$, $Tr(X, X') \triangleq (x' = x - 1 \vee x' = x - 2) \wedge (x > 0 \vee x < -10)$, $X \triangleq \{x\}$. The sink states are $Sink(X) \triangleq x \leq 0 \wedge x \geq -10$. For $n = 2$, quantifier elimination yields $NT^2(X) = x > 2$: states from which $\neg Sink$ is reachable in two transitions.

Step 2: Interpolation (line 2). The following formula is unsatisfiable by construction since any state that satisfies $\neg NT^n$ must reach $Sink$ within at most n steps:

$$\neg NT^n(X^{(0)}) \wedge Tr^n(X^{(0)}, \dots, X^{(n)}) \wedge \neg Sink(X^{(n)})$$

This allows us to compute a *Craig interpolant* that overapproximates the trace Tr^n . Crucially, the usage of the sink states for the interpolant construction allows the interpolant to capture the reason *why* the particular trace is terminating.

Step 3: DNF underapproximation (line 3). For a transition invariant to witness termination, it must be disjunctively well-founded: every disjunct must encode a well-founded relation. Converting the interpolant to DNF would simplify the well-foundedness check for separate disjuncts. However, complete DNF conversion can cause an exponential blowup in formula size and take a significant amount of time. Moreover, even a complete DNF is not guaranteed to yield a transition invariant.

Instead, our algorithm follows a lazy approach: it constructs an *under-approximating* DNF formula on demand. Specifically, the algorithm constructs disjuncts as follows: (1) the SMT solver produces a model that satisfies itp ; (2) the literals satisfied by this model are extracted and conjoined to form a single disjunct; (3) this disjunct is blocked from itp and the process repeats until some m disjuncts are constructed. The result is a formula:

$$Cs(X, X') \triangleq \bigvee_{i=0}^m d_i$$

where each d_i is a conjunction, and $Cs(X, X') \rightarrow itp(X, X')$. This approach significantly reduces formula size and speeds up both the DNF construction and subsequent well-foundedness check, while preserving correctness: in the worst case, Cs fails to be a transition invariant, but this is also possible even with a complete DNF conversion.

Step 4: Filtering Well-Founded Disjuncts (line 4). For each disjunct $d(X, X') \in Cs(X, X')$, the algorithm checks if $d(X, X')$ encodes a well-founded relation. Our technique synthesizes linear ranking functions automatically following [29] for checking the existence of a ranking function. The disjuncts admitting a ranking function are preserved, the rest are discarded. The resulting formula is disjoined with the input candidate transition invariant $TrInv$.

Step 5: Synthesis of the states for which transition invariant is valid (line 5). If $TrInv$ is a transition invariant over all reachable states, then TS terminates, since every disjunct encodes a well-founded relation. To verify this, the procedure computes the formula $C(X)$ encoding the states over which $TrInv$ is a valid transition invariant.

Definition 6 (Coverage formula). Given a transition system $TS = \langle Init, Tr, X \rangle$ and a formula $TrInv(X, X')$, a formula $C(X)$ is called coverage, if the following holds $\forall X, X', X''$:

$$C(X) \wedge Tr(X, X') \rightarrow TrInv(X, X')$$

$$C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X'')$$

That is, $TrInv$ is a transition invariant over states that satisfy $C(X)$.

Similarly to NT^n , $C(X)$ is constructed by applying quantifier elimination, negating the states for which $TrInv$ is not a transition invariant.

Example 3. Continuing Example 2, suppose the procedure constructs $TrInv \triangleq x' < x \wedge x \geq 0$. Quantifier elimination at line 5 then yields $C(X) \triangleq x > 0$.

Note. If $\neg C(X)$ is unreachable, then $TrInv(X, X')$ is a disjunctively well-founded transition invariant over all reachable states, and witnesses termination of the transition system. For illustration, the transition invariant in Example 3 is sufficient to prove termination, since the only states in $\neg C(X) \triangleq x < 0$ that can be reached within TS are sink states.

Lemma 2. Let $TS = \langle Init, Tr, X \rangle$ be a transition system and let $C(X)$ be a formula such that $\neg C(X)$ is unreachable in TS . If $TrInv(X, X')$ is a finite disjunction of formulas encoding well-founded relations and satisfies $\forall X, X', X''$:

$$C(X) \wedge Tr(X, X') \rightarrow TrInv(X, X')$$

$$C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X'')$$

then TS is terminating.

Proof. We show that $TrInv$ is a disjunctively well-founded transition invariant over the reachable states of TS , from which termination follows by [30].

Disjunctive well-foundedness. By construction, for each disjunct d_i of $TrInv = \bigvee_i d_i$ there exists a ranking function. Therefore, every disjunct is a logical encoding of well-founded relation [29]. Hence $TrInv$ is disjunctively well-founded.

Transition invariant. We show that $\forall X, X'. R(X) \wedge Tr^+(X, X') \rightarrow TrInv(X, X')$, where $R(X)$ denotes the reachable states of TS , by induction on the number of steps n in $Tr^+(X, X')$.

Base case ($n = 1$): It is needed to prove that:

$$R(X) \wedge Tr(X, X') \rightarrow TrInv(X, X')$$

Since $\neg C(X)$ is unreachable, $R(X) \rightarrow C(X)$, and therefore $R(X) \wedge Tr(X, X') \rightarrow C(X) \wedge Tr(X, X')$.

The following assumption holds by the lemma statement:

$$C(X) \wedge Tr(X, X') \rightarrow TrInv(X, X').$$

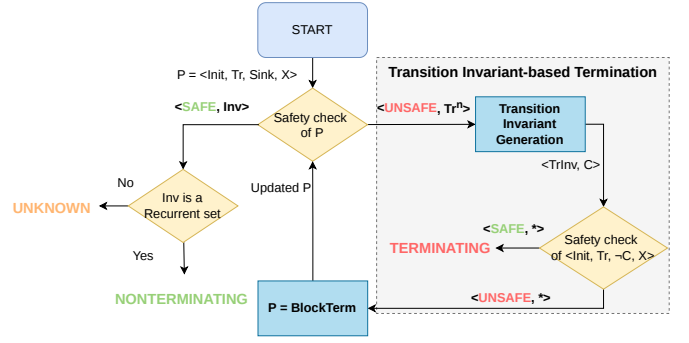


Figure 3. Execution Flow of Interpolation-based Transition Invariant Generation (New components are placed in a grey block, lines 5,6,8).

Therefore the base case holds.

Inductive step: Assume $R(X) \wedge Tr^n(X, \dots, X') \rightarrow TrInv(X, X')$ for some $n \geq 1$. Let $Tr^{n+1}(X, X'')$ hold, so there $\exists X, \dots, X', X''. R(X) \wedge Tr^n(X, \dots, X') \wedge Tr(X', X'')$. Since $R(X) \wedge Tr^n(X, \dots, X') \rightarrow TrInv(X, X')$, then trivially $R(X) \wedge Tr^n(X, X') \wedge Tr(X', X'') \rightarrow R(X) \wedge TrInv(X, X') \wedge Tr(X', X'')$. Now, due to the fact that $R(X) \rightarrow C(X)$, the following holds: $R(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow C(X) \wedge TrInv(X, X') \wedge Tr(X', X'')$. By the lemma hypothesis:

$$C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X'').$$

Therefore, through the chain of implications, we have:

$$R(X) \wedge Tr^{n+1}(X, X') \rightarrow TrInv(X, X').$$

Thus $TrInv$ is a disjunctively well-founded transition invariant over all reachable states, and TS is terminating. $\square$

B. Interpolation-based Termination Analysis

This subsection presents the full termination analysis procedure, using the interpolation-based transition invariant generation. It integrates $TRINVGEN$ into the main loop of Algorithm 1. At each iteration, instead of only blocking terminating states, the algorithm also constructs a transition invariant candidate. The overall flow is shown in Figure 3.

The Algorithm 4 takes as input a safety problem $P = \langle Init, Tr, Sink, X \rangle$, where $Init$ and Tr encode the initial states and transition relation, $Sink$ is the formula encoding sink states, and X is the set of variables over which the transition system is defined. As an output, the algorithm returns $NONTERM$ if TS is nonterminating, $TERM$ if it is terminating, and $UNKNOWN$ if the procedure cannot determine the final result. The procedure maintains a candidate transition invariant $TrInv(X, X')$. During the execution, $TrInv$ is extended with new disjuncts, which are generated by the $TRINVGEN$ procedure, and encode well-founded relations. Changes to the original SNA are at lines 5, 6, and 8.

Every iteration of Algorithm 4, similarly to the Algorithm 1, begins with the safety check at line 2. The algorithm handles two possible cases:

Algorithm 4: Interpolation-based Transition Invariant Generation for Proving and Disproving Termination.

Input : Safety problem $P = \langle \text{Init}, \text{Tr}, \text{Sink}, X \rangle$
Output: TERM | NONTERM | UNKNOWN
Data : TrInv - formula that encodes transition invariant
1 $\text{TrInv} \leftarrow \perp$
2 **while** $\top$ **do**
3 $\langle \text{res}, \phi = \text{Tr}^n \text{ or } \text{Inv} \rangle \leftarrow \text{SAFETYCHECK}(P)$
4 **if** $\text{res} = \text{UNSAFE}$ **then**
5 $\langle \text{TrInv}, C \rangle \leftarrow \text{TRINVGEN}(P, \text{Tr}^n, \text{TrInv})$
6 $\langle \text{res}', _ \rangle \leftarrow \text{SAFETYCHECK}(\langle \text{Init}, \text{Tr}, \neg C, X \rangle)$
7 **if** $\text{res}' = \text{UNSAFE}$ **then** $P \leftarrow \text{BLOCKTERM}(P, \phi)$
8 **else return** TERM
9 **else**
10 **if** $\forall X. \phi(X) \rightarrow \exists X'. \text{Tr}(X, X') \wedge \phi(X')$ **then**
11 **return** NONTERM
12 **return** UNKNOWN

a) **UNSAFE**: A terminating trace Tr^n exists. The algorithm calls TRINVGEN to extend TrInv and compute the coverage formula $C(X)$ (line 5). Then, it checks if $\neg C(X)$ is reachable from the initial states (line 6). If $\neg C(X)$ is reachable, the algorithm refines the safety problem by blocking the states that deterministically lead to termination via BLOCKTERM , and the loop continues. Otherwise, TrInv is a transition invariant over all reachable states, and the algorithm concludes termination.

b) **SAFE**: No sink state is reachable, and there exists an inductive invariant $\text{Inv}(X)$. If $\text{Inv}(X)$ is a recurrent set, the system is nonterminating. Otherwise, the algorithm returns UNKNOWN as the analysis is inconclusive.

Example 4. Consider TS from Example 2 with $\text{Init} \triangleq x > 0$, $\text{Tr} \triangleq (x' = x - 1 \vee x' = x - 2) \wedge (x > 0 \vee x < -10)$, $X \triangleq \{x\}$. Preprocessing computes $\text{Sink} \triangleq x \leq 0 \wedge x \geq -10$. Then, the algorithm is called with $P = \langle \text{Init}, \text{Tr}, \text{Sink}, X \rangle$.

The safety check detects a trace Tr^1 reaching Sink . TRINVGEN produces a transition invariant candidate $\text{TrInv}(X, X') \triangleq x' < x \wedge x' \geq 0$ and the formula $C(X) \triangleq x > 0$. The safety check for $\langle \text{Init}, \text{Tr}, \neg C(X), X \rangle$ returns **SAFE**, and the algorithm concludes that the transition system is terminating.

Limitation of the approach. Consider a different TS , with $X = \{x, u\}$, $\text{Init}(X) \triangleq u < 10 \wedge x > 0$, and $\text{Tr}(X, X') \triangleq ((x' = x - 1 \wedge u' = u) \vee (x' = x + 1 \wedge u' = u + 1 \wedge u < 10)) \wedge x > 0$. Intuitively, each transition either decrements x (and keeps u unchanged), or increments both x and u until $u < 10$. Once $u = 10$, only the decrementing transition is possible, and x decreases monotonically to zero. Hence TS is terminating. The sink states are $\text{Sink}(X) \triangleq x \leq 0$.

The Algorithm 4 detects a trace $\text{Tr}^1(X^{(0)}, X^{(1)})$ reaching Sink states. TRINVGEN constructs transition invariant: $\text{TrInv}(X, X') \triangleq x' < x \wedge x > 0$. The coverage formula is computed as: $C(X) \triangleq u = 10$. The safety check for $\langle \text{Init}, \text{Tr}, \neg C, X \rangle$ returns **UNSAFE**, since states with $u < 10$ are reachable from Init . BLOCKTERM , then blocks the states that are guaranteed to reach sink: $\text{Tr}(X, X') \leftarrow$

$\text{Tr}(X, X') \wedge x' > 0$.

The algorithm calls $\text{SAFETYCHECK}(P)$, which returns **SAFE** with inductive invariant $\text{Inv}(X) \triangleq x > 0$. However, Inv is not a recurrent set, so Algorithm 4 returns **UNKNOWN**.

The root cause why the algorithm cannot prove termination is that TrInv does not cover states with $u < 10$. It captures why the system terminates when $u = 10$, but not the global argument: u grows until at most $u = 10$, after which x decreases monotonically. A complete termination proof requires a *second* disjunct covering the $u < 10$ region. In Section IV, we present an extension that focuses on the state space for which the transition invariant candidate is not valid, aiming to extend the transition invariant.

C. Correctness

This subsection proves the soundness of the introduced approach. We first prove that when the algorithm returns **TERM**, the transition system is terminating, and then we prove the correctness for the nontermination.

Lemma 3. *If Algorithm 4 returns **TERM**, then the transition system $\langle \text{Init}, \text{Tr}, X \rangle$ is terminating.*

Proof. **TERM** is returned only at line 8. It means that there exists a transition invariant TrInv which is valid over states $C(X)$, and $\neg C(X)$ is unreachable from the initial states. The safety problem is updated iteratively, blocking the states that deterministically lead to the termination, so in the end the $P = \langle \text{Init}', \text{Tr}', \text{Sink}, X \rangle$, where $\text{Init}'(X) = \text{Init}(X) \wedge \bigwedge_{i=0}^m \neg S_i(X)$ and $\text{Tr}' = \text{Tr}(X, X') \wedge \bigwedge_{i=0}^l \neg S_j(X')$. According to Lemma 2, transition system $\langle \text{Init}', \text{Tr}', X \rangle$ is terminating, since TrInv is a disjunctively well-founded transition invariant over all reachable states.

Since the states satisfying $\bigvee_{i=0}^m S_i(X)$ and $\bigvee_{i=0}^l S_j(X')$ deterministically lead to the termination, every trace of $\langle \text{Init}, \text{Tr}, X \rangle$ either follows Tr' or passes through a blocked state which leads to termination. Hence $\langle \text{Init}, \text{Tr}, X \rangle$ is terminating. $\square$

Lemma 4. *If Algorithm 4 returns **NONTERM**, then the transition system $\langle \text{Init}, \text{Tr}, X \rangle$ is nonterminating.*

Proof. The proof of this lemma follows directly from Lemma 1, as the nontermination analysis in Algorithm 4 follows exactly the same structure as Algorithm 1. $\square$

IV. EXTENDED INTERPOLATION-BASED TRANSITION INVARIANT GENERATION FOR PROVING AND DISPROVING TERMINATION

Algorithm 4 is already capable of proving termination efficiently through the construction of transition invariants. This section describes an extension of the algorithm that aims to achieve better interaction between termination and nontermination analysis. Additionally, we present further extensions to the original SNA algorithm that improve overall performance. The high-level flow of the extended algorithm can be seen in Figure 4.

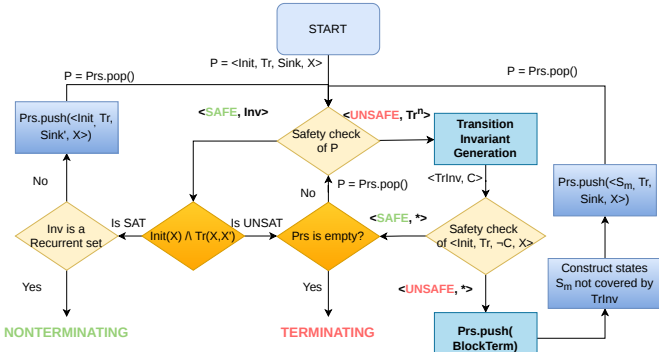


Figure 4. Extended Interpolation-based Termination Analysis (New components are highlighted in dark orange and dark blue colors).

A. Utilizing States for Which Transition Invariant is not Valid

A transition invariant candidate can be used to guide the termination analysis. Particularly, Algorithm 4 constructs the formula $C(X)$ encoding states for which the transition invariant is valid. If $\neg C(X)$ is reachable, there exist states for which the transition invariant is not valid, and which potentially can lead to nontermination. The intuition is to compute the reachable states S_m within $\neg C(X)$ (which corresponds to line 11 of Algorithm 5). Then, it is possible to run (non)termination check for these particular states within TS (line 12).

In the case of termination, the algorithm produces a witness transition invariant that extends the transition invariant of the original system, making it valid for more states. If nontermination is proved for S_m , then the original TS is also nonterminating, since S_m is reachable from the initial states.

B. Overall Algorithm

Algorithm 5 extends Algorithm 4 to enable more efficient (non)termination analysis. First, in addition to maintaining the transition invariant, the algorithm maintains a stack of safety problems to be analyzed: Prs (line 2). At each iteration, the algorithm pops a safety problem from the stack and analyses it. The stack allows the algorithm to focus on (non)termination analysis of particular reachable states within TS . The original safety problem always remains in the Prs stack.

The postprocessing phase of transition invariant construction is the main change to Algorithm 4. When $\neg C$ is reachable, the extended algorithm performs two actions: (1) it pushes the BLOCKTERM-refined problem (line 10); (2) it computes a formula encoding states reachable within transition system, for which transition invariant candidate is not valid (line 11); and finally, it pushes a new subproblem to analyse for termination (line 12), with S_m as the initial states. This allows the algorithm to concentrate on proving termination or nontermination of particular reachable states.

If $\neg C$ is not reachable, $TrInv$ proves termination of the currently analyzed TS (the last TS popped from the stack), and the number of remaining subproblems decreases. If the

Algorithm 5: Extended Interpolation-based Transition Invariant Generation for Proving and Disproving Termination.

Input : Safety problem P
Output: TERM | NONTERM
Data : $TrInv$ - formula that encodes transition invariant,
 Prs - stack of Safety Problems to be analyzed for (non)termination

```

1  $TrInv \leftarrow \perp$ 
2  $Prs \leftarrow [P]$ 
3 while  $\top$  do
4    $P' \equiv \langle Init, Tr, Sink, X \rangle \leftarrow Prs.pop()$ 
5    $\langle res, \phi \rangle \leftarrow \text{SAFETYCHECK}(\langle Init, Tr, Sink, X \rangle)$ 
6   if  $res = \text{UNSAFE}$  then
7      $\langle TrInv, C \rangle \leftarrow \text{TRINVGEN}(P', \phi, TrInv)$ 
8      $\langle res', \phi' \rangle \leftarrow \text{SAFETYCHECK}(\langle Init, Tr, \neg C, X \rangle)$ 
9     if  $\langle res', \phi' \rangle = \langle \text{UNSAFE}, Tr^m \rangle$  then
10       $Prs.push(\text{BLOCKTERM}(P', \phi))$ 
11       $S_m(X) \leftarrow QE(\exists X^{(0)}, \dots, X^{(m-1)}. Init(X^{(0)}) \wedge$ 
12         $\phi'(X^{(0)}, \dots, X^{(m)}) \wedge \neg C(X^{(m)})) [X^{(m)} \mapsto X]$ 
13       $Prs.push(\langle S_m, Tr, C, X \rangle)$ 
14   else
15     if  $Prs = \emptyset$  then return TERM
16     else continue
17   else
18     if  $\forall X. \phi(X) \rightarrow \exists X'. Tr(X, X') \wedge \phi(X')$  then
19       return NONTERM
20     if  $\text{UNSAT? } \phi(X) \wedge Tr(X, X')$  then
21       if  $Prs = \emptyset$  then return TERM
22       else continue
23      $Sink'(X) \leftarrow \neg QE(\exists X'. Tr(X, X'))$ 
24      $Prs.push(\langle Init, Tr, Sink', X \rangle)$ 

```

stack is empty, Algorithm 5 returns TERM. The algorithm incorporates two further extensions.

Simple termination check (line 20). When the safety check returns SAFE and $\phi(X)$ is not a recurrent set, the algorithm additionally checks whether it is possible to take a transition from the initial states by querying:

$$Init(X) \wedge Tr(X, X')$$

If this formula is unsatisfiable, then no transition can be taken from the initial states (or all transitions are blocked, meaning all reachable states deterministically lead to termination), and the system is trivially terminating.

Refinement of Sink states (lines 22, 23). When $\phi(X)$ is not a recurrent set, this is often because BLOCKTERM has eliminated all transitions leading to $Sink$. This means that the produced ϕ contains sink states due to the updated Tr . New formula encoding sink states is computed as:

$$Sink'(X) = \neg QE(\exists X'. Tr'(X, X'))$$

The formula $Sink'$ replaces $Sink$ in the safety problem, enabling the analysis to proceed instead of returning UNKNOWN.

Lemma 5. *If Algorithm 5 returns TERM, then the original transition system $\langle Init, Tr, X \rangle$ is also terminating.*

Proof. TERM is returned at line 14 or line 20. As in Algorithm 4, to return TERM, the termination conditions need to be satisfied for the transition system $\langle \text{Init}', \text{Tr}', X \rangle$, where $\text{Init}'(X) = \text{Init}(X) \wedge \bigwedge_i \neg S_i(X)$, and $\text{Tr}'(X, X') = \text{Tr}(X, X') \wedge \bigwedge_j \neg S_j(X')$. $\bigvee_{i=0}^m S_i(X)$ and $\bigvee_{j=0}^l S_j(X')$ are the states that deterministically lead to termination.

Case 1. If TERM is returned at line 14, then the termination proof follows directly from the Lemma 3. This is the case, since the original safety problem is preserved in the stack, so if $\text{Prs} = \emptyset$, the original safety problem is necessarily terminating.

Case 2. If TERM is returned at line 20, then the formula $\text{Init}'(X) \wedge \text{Tr}'(X, X')$ is unsatisfiable, so no transition is reachable from Init' . This means that the transition system is terminating, since all feasible transitions in the original transition system lead to deterministically terminating states. $\square$

Lemma 6. *If Algorithm 5 returns NONTERM, then the original transition system $\langle \text{Init}, \text{Tr}, X \rangle$ is nonterminating.*

Proof. NONTERM is returned at line 18, where $\phi(X)$ is a recurrent set for the current refined system $\langle \text{Init}', \text{Tr}', X \rangle$. By construction, Init' is reachable from Init (or it is Init), and $\text{Tr}'(X, X') \rightarrow \text{Tr}(X, X')$. By Lemma 1, if NONTERM is returned, then there exists a recurrent set in the transition system $\langle \text{Init}', \text{Tr}', X \rangle$, which means that there also exists a recurrent set in the original transition system $\langle \text{Init}, \text{Tr}, X \rangle$. This proves nontermination. $\square$

V. IMPLEMENTATION AND EVALUATION

This section presents the implementation details and experimental evaluation of our approach.

A. Implementation

We implemented the prototypes of Algorithms 1, 4, and 5 inside of GOLEM; we refer to these implementations as SNA, IPTTIG and IPTTIG+. GOLEM was chosen because it supports multiple different safety verification procedures, such as Property-Directed Reachability [7] (PDR), Transition Power Abstraction [6] (TPA), and Interpolation-based Model Checking [28]. Additionally, we used quantifier elimination, already implemented in GOLEM.

For SMT queries and interpolant construction, our implementation uses the OPENSMT SMT solver [11], [21]. OPENSMT is integrated within GOLEM and provides native support for interpolation.

B. Evaluation

We compared IPTTIG+ against SNA and IPTTIG to evaluate the effect of combining termination and nontermination analysis. Additionally, we evaluated IPTTIG+ against several state-of-the-art tools: LOAT [27] (version each74b), KOAT [27] (version 41cb681), T2 [9] (version 10f1373). The evaluation focused on termination analysis of Integer Transition Systems. Experiments were conducted on Ubuntu 20.04 machine with an AMD EPYC 7452 32-core processor and 8x32 GiB RAM. Our evaluation addressed the following research questions:

Table I
COMPARISON OF IPTTIG+ WITH SNA AND IPTTIG.

Algorithm	Total	Terminating	Nonterminating	Uniquely
SNA	343	0	343	7
IPTTIG	521	186	335	7
IPTTIG+	761	320	441	240

Table II
COMPARISON OF IPTTIG+ WITH STATE-OF-THE-ART TOOLS.

Algorithm	Total	Terminating	Nonterminating	Uniquely
LOAT	525	0	525	48
KOAT	562	562	0	26
T2	1002	572	430	40
IPTTIG+	761	320	441	8

- **RQ1:** How does extending SNA with the termination procedure affect verification performance?
- **RQ2:** How does IPTTIG+ perform compared to state-of-the-art tools?

The benchmark suite of Integer Transition Systems was taken from the Termination Competition [19]. Overall, there are 1222 benchmarks (538 nonterminating, 665 terminating, and 19 unknown). We excluded 42 trivially terminating benchmarks from the evaluation (termination of which can be detected by trivial syntactic checks), which are solved by all tools. The evaluation was executed with the 120 seconds timeout (the increase of the timeout does not significantly influence the evaluation).

Table I presents the experimental results for IPTTIG+, SNA, and IPTTIG. Our results indicate that IPTTIG+ consistently outperforms both SNA and IPTTIG for terminating and non-terminating instances, solving 240 instances not solved by neither SNA nor IPTTIG. We attribute this to the IPTTIG+'s generation of focused sub-problems, which allows the analysis to concentrate on specific reachable states. This benefits the recurrent set detection by focusing the analysis on specific states not covered by the transition invariant. It also benefits termination proving by extending the transition invariant with additional disjuncts that cover different regions of the reachable state space. In answer to **RQ1**, the combination of termination and nontermination reasoning in IPTTIG+ yields significantly improved performance across both problem classes. Notably, during experimentation we noticed that IPTTIG+ produces different outputs depending on the underlying Safety Verification engine. The best results were obtained with the TPA safety checks, solving 761 instances. IPTTIG+ with PDR solved fewer instances (756), however it could solve 17 benchmarks uniquely compared to the TPA-based algorithm.

Figure 5 shows a scatter plot comparing IPTTIG+ against state-of-the-art termination analysers, where each axis represents the solving time in seconds for a given tool. Orange squares denote nonterminating instances; blue crosses denote terminating instances. Points above the diagonal indicate instances where IPTTIG+ is faster than the competitor. IPTTIG+ solves 761 benchmarks, with T2 solving 1002, KOAT 562, and LOAT 525 instances, the full comparison

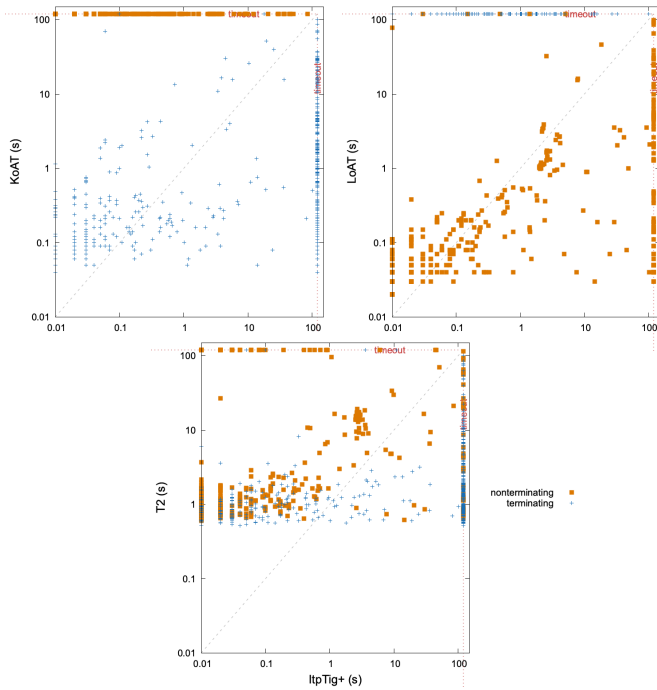


Figure 5. Comparison of ITPTIG+ with state-of-the-art tools.

can be seen in Table II. As shown in Figure 5, ITPTIG+, despite being a prototype implementation, is competitive with state-of-the-art termination analysers. Among all tools, ITPTIG+ uniquely solves 8 instances (those instances were not solved by SNA and ITPTIG as well). Notably, 2 of these — `juLinkedListCreateAddAll.jar-obl-11` and `juLinkedListCreateAddAllAt.jar-obl-17` — belong to the 19 previously unsolved instances in the Termination Competition, for which ITPTIG+ found recurrent sets thus leading to proving nontermination. In answer to **RQ2**, ITPTIG+ is competitive with state-of-the-art tools overall and uniquely solves instances previously unsolved by any tool in the Termination Competition.

VI. RELATED WORK

Termination and nontermination analysis are active areas of research. Below we discuss the techniques most closely related to our approach.

Nontermination. Nontermination analysis is typically centered on the construction of recurrent sets — sets of states from which execution cannot escape [9], [10], [12], [16], [17], [24]. These techniques range from template-based synthesis to safety-check-based recurrent set construction, the latter of which our approach directly extends [12]. An alternative approach detects nontermination by identifying lasso-shaped executions — finite paths leading to a cycle — via bounded model checking [4]. Leike and Heizmann [26] propose geometric nontermination witnesses: finite representations of infinite executions via sums of geometric series. Compared to the approaches above, our technique is integrated with termination

analysis within a unified framework, so that intermediate results from each analysis guide the other.

Termination. The most widely studied approach to termination analysis is the synthesis of ranking functions [2], [16], [18], [23], [29], [31], [32], [35]. State-of-the-art techniques focus on the construction of multiphase or lexicographic ranking functions [2], [16], which significantly broaden the class of programs for which termination can be proved. However, the complexity of a transition system can make ranking function synthesis infeasible. An alternative is fixpoint computation [34], which reduces termination and nontermination to the validity of a first-order fixpoint formula. Another family of techniques proves termination via disjunctively well-founded transition invariants [13], [22], [30], [33]. These approaches over-approximate the behavior of the transition system, reducing the termination proof to a disjunctive well-foundedness check on the invariant. The central challenge is the generation of the transition invariant itself: existing techniques rely either on template-based generation or on the direct construction of a ranking relation from a particular trace. In contrast, our approach uses Craig interpolation to guide transition invariant synthesis with respect to the actual termination conditions, producing property-directed invariants from terminating traces.

VII. CONCLUSION

We have presented an interpolation-based approach to transition invariant generation for proving and disproving termination of infinite-state systems. The approach extends safety-based nontermination analysis with a termination proving capability, unifying the two within a single framework. Our key insight is to exploit terminating traces produced by the SNA as a source of structural information, using Craig interpolation to construct well-founded transition invariants guided by the termination conditions. The interaction between termination and nontermination reasoning is central to our technique: transition invariants focus the nontermination analysis on not covered states, while SNA provides terminating traces that drive invariant generation. Grounding the approach in safety verification provides a robust foundation, enabling efficient checking of both termination and nontermination via existing safety solvers. We proved the soundness of the algorithm and conducted an experimental evaluation on benchmarks from the Termination Competition. The results demonstrate the benefit of combining termination and nontermination analysis, and show that our approach achieves competitive performance against state-of-the-art tools, including the unique solution of instances previously unsolved by any tool in the Termination Competition.

Acknowledgements. This work was supported in part by SNSF grant No. 200021-236601. We thank Marek Jankola and Dirk Beyer for fruitful discussions on termination analysis.

REFERENCES

- [1] Ben-Amram, A.M., Doménech, J.J., Genaim, S.: Multiphase-linear ranking functions and their relation to recurrent sets. In: Chang, B.E. (ed.) Static Analysis - 26th International Symposium, SAS 2019, Porto,

- Portugal, October 8-11, 2019, Proceedings. pp. 459–480. Lecture Notes in Computer Science, Springer (2019). https://doi.org/10.1007/978-3-030-32304-2_22
- [2] Ben-Amram, A.M., Genaim, S.: Ranking functions for linear-constraint loops. *J. ACM* **61**(4), 26:1–26:55 (2014). <https://doi.org/10.1145/2629488>, <https://doi.org/10.1145/2629488>
 - [3] Beyer, D., Jankola, M., Lingsch-Rosenfeld, M.: Transition invariants revisited: Termination witnesses and their validation. In: Proc. CAV. Springer (2026). https://www.sosy-lab.org/research/pub/2026-CAV-Transition_Invariants_Revisited_Termination_Witnesses_and_Their_Validation.pdf
 - [4] Biere, A., Artho, C., Schuppan, V.: Liveness checking as safety checking. In: Cleaveland, R., Garavel, H. (eds.) 7th International ERCIM Workshop in Formal Methods for Industrial Critical Systems, FMICS 2002, ICALP 2002 Satellite Workshop, Málaga, Spain, July 12-13, 2002. pp. 160–177. No. 2 in Electronic Notes in Theoretical Computer Science, Elsevier (2002). [https://doi.org/10.1016/S1571-0661\(04\)80410-9](https://doi.org/10.1016/S1571-0661(04)80410-9), [https://doi.org/10.1016/S1571-0661\(04\)80410-9](https://doi.org/10.1016/S1571-0661(04)80410-9)
 - [5] Blichla, M., Britikov, K., Sharygina, N.: The golem horn solver. In: Enea, C., Lal, A. (eds.) Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part II. Lecture Notes in Computer Science, vol. 13965, pp. 209–223. Springer (2023). https://doi.org/10.1007/978-3-031-37703-7_10
 - [6] Blichla, M., Fedyukovich, G., Hyvärinen, A.E.J., Sharygina, N.: Transition power abstractions for deep counterexample detection. In: Fisman, D., Rosu, G. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022. Lecture Notes in Computer Science, vol. 13243, pp. 524–542. Springer (2022). https://doi.org/10.1007/978-3-030-99524-9_29
 - [7] Bradley, A.R., Manna, Z.: Property-directed incremental invariant generation. *Formal Aspects Comput.* **20**(4-5), 379–405 (2008). <https://doi.org/10.1007/S00165-008-0080-9>, <https://doi.org/10.1007/s00165-008-0080-9>
 - [8] Braverman, M.: Termination of integer linear programs. In: Ball, T., Jones, R.B. (eds.) Computer Aided Verification, 18th International Conference, CAV 2006, Seattle, WA, USA, August 17-20, 2006, Proceedings. pp. 372–385. Lecture Notes in Computer Science, Springer (2006). https://doi.org/10.1007/11817963_34, https://doi.org/10.1007/11817963_34
 - [9] Brockschmidt, M., Cook, B., Ishtiaq, S., Khlaaf, H., Piterman, N.: T2: temporal property verification. In: Chechik, M., Raskin, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings. pp. 387–393. Lecture Notes in Computer Science, Springer (2016). https://doi.org/10.1007/978-3-662-49674-9_22, https://doi.org/10.1007/978-3-662-49674-9_22
 - [10] Brockschmidt, M., Ströder, T., Otto, C., Giesl, J.: Automated detection of non-termination and nullpointerexceptions for java bytecode. In: Beckert, B., Damiani, F., Gurov, D. (eds.) Formal Verification of Object-Oriented Software - International Conference, FoVeOS 2011, Turin, Italy, October 5-7, 2011, Revised Selected Papers. pp. 123–141. Lecture Notes in Computer Science, Springer (2011). https://doi.org/10.1007/978-3-642-31762-0_9, https://doi.org/10.1007/978-3-642-31762-0_9
 - [11] Bruttomesso, R., Pek, E., Sharygina, N., Tsitovich, A.: The opensmt solver. In: Esparza, J., Majumdar, R. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, 16th International Conference, TACAS 2010, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2010, Paphos, Cyprus, March 20-28, 2010. Proceedings. pp. 150–153. Lecture Notes in Computer Science, Springer (2010). https://doi.org/10.1007/978-3-642-12002-2_12, https://doi.org/10.1007/978-3-642-12002-2_12
 - [12] Chen, H.Y., Cook, B., Fuhs, C., Nimkar, K., O’Hearn, P.W.: Proving nontermination via safety. In: Ábrahám, E., Havelund, K. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014. Proceedings. pp. 156–171. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-642-54862-8_11, https://doi.org/10.1007/978-3-642-54862-8_11
 - [13] Cook, B., Podelski, A., Rybalchenko, A.: Abstraction refinement for termination. In: Hankin, C., Siveroni, I. (eds.) Static Analysis, 12th International Symposium, SAS 2005, London, UK, September 7-9, 2005, Proceedings. pp. 87–101. Lecture Notes in Computer Science, Springer (2005). https://doi.org/10.1007/11547662_8, https://doi.org/10.1007/11547662_8
 - [14] Cook, B., Podelski, A., Rybalchenko, A.: Termination proofs for systems code. In: Schwartzbach, M.I., Ball, T. (eds.) Proceedings of the ACM SIGPLAN 2006 Conference on Programming Language Design and Implementation, Ottawa, Ontario, Canada, June 11-14, 2006. pp. 415–426. ACM (2006). <https://doi.org/10.1145/1133981.1134029>, <https://doi.org/10.1145/1133981.1134029>
 - [15] Craig, W.: Three uses of the Herbrand-Gentzen theorem in relating model theory and proof theory. *The Journal of Symbolic Logic* **22**(3), 269–285 (1957)
 - [16] Fedyukovich, G., Zhang, Y., Gupta, A.: Syntax-guided termination analysis. In: Chockler, H., Weissenbacher, G. (eds.) Computer Aided Verification - 30th International Conference, CAV 2018, Held as Part of the Federated Logic Conference, FloC 2018, Oxford, UK, July 14-17, 2018, Proceedings, Part I. pp. 124–143. Lecture Notes in Computer Science, Springer (2018). https://doi.org/10.1007/978-3-319-96145-3_7, https://doi.org/10.1007/978-3-319-96145-3_7
 - [17] Frohn, F., Giesl, J.: Proving non-termination and lower runtime bounds with loat (system description). In: Blanchette, J., Kovács, L., Pattinson, D. (eds.) Automated Reasoning - 11th International Joint Conference, IJCAR 2022, Haifa, Israel, August 8-10, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13385, pp. 712–722. Springer (2022). https://doi.org/10.1007/978-3-031-10769-6_41, https://doi.org/10.1007/978-3-031-10769-6_41
 - [18] Giesl, J., Aschermann, C., Brockschmidt, M., Emmes, F., Frohn, F., Fuhs, C., Hensel, J., Otto, C., Plücker, M., Schneider-Kamp, P., Ströder, T., Swiderski, S., Thiemann, R.: Analyzing program termination and complexity automatically with approve. *J. Autom. Reason.* **58**(1), 3–31 (2017). <https://doi.org/10.1007/S10817-016-9388-Y>, <https://doi.org/10.1007/s10817-016-9388-y>
 - [19] Giesl, J., Mesnard, F., Rubio, A., Thiemann, R., Waldmann, J.: Termination competition (termcomp 2015). In: Felty, A.P., Middeldorp, A. (eds.) Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings. pp. 105–108. Lecture Notes in Computer Science, Springer (2015). https://doi.org/10.1007/978-3-319-21401-6_6, https://doi.org/10.1007/978-3-319-21401-6_6
 - [20] Heizmann, M., Hoenicke, J., Podelski, A.: Termination analysis by learning terminating programs. In: Biere, A., Bloem, R. (eds.) Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings. pp. 797–813. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_53, https://doi.org/10.1007/978-3-319-08867-9_53
 - [21] Hyvärinen, A.E.J., Marescotti, M., Alt, L., Sharygina, N.: Opensmt2: An SMT solver for multi-core and cloud computing. In: Creignou, N., Berre, D.L. (eds.) Theory and Applications of Satisfiability Testing - SAT 2016 - 19th International Conference, Bordeaux, France, July 5-8, 2016, Proceedings. Lecture Notes in Computer Science, vol. 9710, pp. 547–553. Springer (2016). https://doi.org/10.1007/978-3-319-40970-2_35, https://doi.org/10.1007/978-3-319-40970-2_35
 - [22] Kroening, D., Sharygina, N., Tsitovich, A., Wintersteiger, C.M.: Termination analysis with compositional transition invariants. In: Touili, T., Cook, B., Jackson, P.B. (eds.) Computer Aided Verification, 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings. pp. 89–103. Lecture Notes in Computer Science, Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_9, https://doi.org/10.1007/978-3-642-14295-6_9
 - [23] Kura, S., Unno, H., Hasuo, I.: Decision tree learning in cegis-based termination analysis. In: Silva, A., Leino, K.R.M. (eds.) Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II. pp. 75–98. Lecture Notes in Computer Science, Springer (2021). https://doi.org/10.1007/978-3-030-81688-9_4, https://doi.org/10.1007/978-3-030-81688-9_4
 - [24] Larraz, D., Nimkar, K., Oliveras, A., Rodríguez-Carbonell, E., Rubio, A.: Proving non-termination using max-smt. In: Biere, A., Bloem, R. (eds.) Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings. pp. 779–796. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_52, https://doi.org/10.1007/978-3-319-08867-9_52

- [25] Le, T.C., Qin, S., Chin, W.: Termination and non-termination specification inference. In: Grove, D., Blackburn, S.M. (eds.) *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, Portland, OR, USA, June 15-17, 2015. pp. 489–498. ACM (2015). <https://doi.org/10.1145/2737924.2737993>, <https://doi.org/10.1145/2737924.2737993>
- [26] Leike, J., Heizmann, M.: Geometric nontermination arguments. In: Beyer, D., Huisman, M. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems - 24th International Conference, TACAS 2018, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2018, Thessaloniki, Greece, April 14-20, 2018, Proceedings, Part II*. pp. 266–283. *Lecture Notes in Computer Science*, Springer (2018). https://doi.org/10.1007/978-3-319-89963-3_16, https://doi.org/10.1007/978-3-319-89963-3_16
- [27] Lommen, N., Giesl, J.: Aprove (koat + loat). In: Gurfinkel, A., Heule, M. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems*. pp. 205–211. Springer Nature Switzerland, Cham (2025)
- [28] McMillan, K.L.: Interpolation and sat-based model checking. In: Jr., W.A.H., Somenzi, F. (eds.) *Computer Aided Verification, 15th International Conference, CAV 2003, Boulder, CO, USA, July 8-12, 2003, Proceedings*. *Lecture Notes in Computer Science*, vol. 2725, pp. 1–13. Springer (2003). https://doi.org/10.1007/978-3-540-45069-6_1
- [29] Podelski, A., Rybalchenko, A.: A complete method for the synthesis of linear ranking functions. In: Steffen, B., Levi, G. (eds.) *Verification, Model Checking, and Abstract Interpretation, 5th International Conference, VMCAI 2004, Venice, Italy, January 11-13, 2004, Proceedings*. pp. 239–251. *Lecture Notes in Computer Science*, Springer (2004). https://doi.org/10.1007/978-3-540-24622-0_20, https://doi.org/10.1007/978-3-540-24622-0_20
- [30] Podelski, A., Rybalchenko, A.: Transition invariants. In: *19th IEEE Symposium on Logic in Computer Science (LICS 2004)*, 14-17 July 2004, Turku, Finland, *Proceedings*. pp. 32–41. IEEE Computer Society (2004). <https://doi.org/10.1109/LICS.2004.1319598>, <https://doi.org/10.1109/LICS.2004.1319598>
- [31] Riley, D., Fedyukovich, G.: Exact loop bound analysis. *Proc. ACM Program. Lang.* **9**(PLDI), 1814–1837 (2025). <https://doi.org/10.1145/3729323>, <https://doi.org/10.1145/3729323>
- [32] Sarita, Y., Singh, A., Gomber, S., Singh, G., Vishwanathan, M.: Efficient ranking function-based termination analysis via bidirectional decomposition search. In: Krebbers, R. (ed.) *Programming Languages and Systems - 35th European Symposium on Programming, ESOP 2026, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2026, Turin, Italy, April 11-16, 2026, Proceedings, Part II*. pp. 181–209. *Lecture Notes in Computer Science*, Springer (2026). https://doi.org/10.1007/978-3-032-22723-2_7, https://doi.org/10.1007/978-3-032-22723-2_7
- [33] Tsitovich, A., Sharygina, N., Wintersteiger, C.M., Kroening, D.: Loop summarization and termination analysis. In: Abdulla, P.A., Leino, K.R.M. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems - 17th International Conference, TACAS 2011, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2011, Saarbrücken, Germany, March 26-April 3, 2011, Proceedings*. pp. 81–95. *Lecture Notes in Computer Science*, Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_9, https://doi.org/10.1007/978-3-642-19835-9_9
- [34] Unno, H., Terauchi, T., Gu, Y., Koskinen, E.: Modular primal-dual fixpoint logic solving for temporal verification. *Proc. ACM Program. Lang.* **7**(POPL), 2111–2140 (2023). <https://doi.org/10.1145/3571265>, <https://doi.org/10.1145/3571265>
- [35] Urban, C., Gurfinkel, A., Kahsai, T.: Synthesizing ranking functions from bits and pieces. In: Chechik, M., Raskin, J. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems - 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings*. pp. 54–70. *Lecture Notes in Computer Science*, Springer (2016). https://doi.org/10.1007/978-3-662-49674-9_4, https://doi.org/10.1007/978-3-662-49674-9_4